

Probability simulations with Python

Uniform random integers and floats

- Basic probability simulations are based on use of random number generators and counting
- Python has high quality random number generators
- `Generator` class from `numpy.random` is a modern implementation
- Uniform random values means that in a certain interval all values are equally likely

```
from numpy.random import default_rng
rng = default_rng()
ints = rng.integers(low=1, high=10, size=(1,15)) #low is included, but high is excluded
print(f"Random integers 1...9: {ints}")

flts = rng.random(size = (1,10)) #floats 0...1
print(f"Random floats between 0 ... 1: {flts}")

a = 2 #scaling factor, from 0...1 to 0...a
b = 1 #shift, from 0...a to b...a+b
flts2 = a*rng.random(size = (1,10)) + b
print(f"Random floats between 1 ... 3: {flts2}")
```

```
Random integers 1...9: [[7 8 8 6 8 7 1 3 3 1 1 2 9 7 4]]
Random floats between 0 ... 1: [[0.56169189 0.97587006 0.82963611 0.57539796 0.89762662 0.88784491
0.03637164 0.21816785 0.24739032 0.52796642]]
Random floats between 1 ... 3: [[1.92447129 2.82323224 1.44075581 2.38061836 2.09891982 2.62076546
2.01350361 1.45718345 2.17445581 2.67865114]]
```

Random samples with `choice`

- The function `choice` from `numpy` generates a random sample from a given population (typically a list of values).
- It is possible to sample in two ways: A certain element can be selected more than once (**with replacement**) or a certain element can only be selected once **without replacement**
- When sampling with replacement the next result is **dependent of the previous one(s)**. This is because some element(s) will be missing.
- If sampling is made with replacement the next result is **independent of the previous one(s)**.

Example 1

Assume that you have a bag of 10 marbles of known colors. You draw a marble from the bag and you notice the color of the marble.

- **with replacement:** you put the drawn marble back to the bag. There would be 10 marbles in the next draw. Knowledge of the first result has no effect on the probability of the next draw result.
- **without replacement:** you don't put the marble back to the back. There would be only 9 marbles in the next draw. Knowledge of the first result has an effect on the probability of the next draw result.

Example 2

1. In lottery seven numbers are sampled out of 40. This sampling is **without replacement**, because the same number is not allowed twice.
2. When tossing a coin several times the sampling is **with replacement** because both head and tail are possible in all tosses.

Example, tossing a fair coin

```
import numpy as np

# Number of coin tosses to simulate
num_tosses = 1_000_000

rng = np.random.default_rng()

tosses = rng.choice(['H', 'T'], size=num_tosses)

# Calculate the probabilities
heads_prob = np.sum(tosses == 'H') / num_tosses
tails_prob = 1 - heads_prob

print(f"Probability of Heads: {heads_prob:.5f}")
print(f"Probability of Tails: {tails_prob:.5f}")
```

```
Probability of Heads: 0.50056
Probability of Tails: 0.49944
```

Example, tossing a biased coin

```
from numpy.random import default_rng
from numpy import sum

bias = 0.6 # 60% chance of heads, 40% chance of tails
num_tosses = 1_000_000

# Create a random number generator
rng = default_rng()
tosses = rng.choice(['Heads', 'Tails'], size=num_tosses, p=[bias, 1 - bias])

heads_prob = sum(tosses == 'Heads') / num_tosses
tails_prob = sum(tosses == 'Tails') / num_tosses

print(f"Probability of Heads: {heads_prob:.5f}")
print(f"Probability of Tails: {tails_prob:.5f}")
```

```
Probability of Heads: 0.59989
Probability of Tails: 0.40011
```


Simulation of a deck of cards

- `itertools` can be used to build a deck of cards
 - `itertools.product` forms all possible pairs of elements of two lists.
- To build a deck of cards strings corresponds to suit and integers or strings correspond to values. (The value could of course also be a string)
- `itertools.product` is a lot faster than nested `for` loops to form the pairs

```
from itertools import product
deck = list(product(['He', 'Cl', 'Di', 'Sp'], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]))
print(deck)
```

Example, probability of drawing Ace of Hearts or Ace of Diamonds when two cards are drawn

```
from numpy.random import default_rng
from itertools import product

deck = list(product(['He', 'Cl', 'Di', 'Sp'], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]))
rng = default_rng()
n_shuffles = 100_000
count = 0

for i in range(n_shuffles):
    cards_drawn = rng.choice(deck, size = 2, replace = False)
    for card in cards_drawn:
        if all(card == ['Di', '1']) or all(card == ['He', '1']):
            count += 1
print(f"P = {count/n_shuffles}")
print(f"Exact: P = {1 - 50/52*49/51:.5f}")
```

```
P = 0.07646
Exact: P = 0.07617
```

Contingency tables

- Contingency tables can be formed using `pandas.crosstab`
- The example is from Geraghty

```
import pandas as pd
data = {
    'DUI_Status': ['DUI']*200 + ['Non-DUI']*800,
    'Accident_Status': (['Accident']*70 + ['No accident']*130 +
                        ['Accident']*30 + ['No accident']*770)
}
df = pd.DataFrame(data)
# Create contingency table
table = pd.crosstab(df['DUI_Status'], df['Accident_Status'], margins=True)
print(table)
total = table.loc['All', 'All']
print(" ")
print("P(DUI) =", table.loc['DUI', 'All']/total)
print("P('DUI and Accident') =", table.loc['DUI', 'Accident']/total)
print("P('DUI and No accident')=", table.loc['DUI', 'No accident']/total)
print("P('Non-DUI and Accident')=", table.loc['Non-DUI', 'Accident']/total)
print("P('DUI given Accident') =", table.loc['DUI', 'Accident'] / table.loc['All', 'Accident'])
print("P('Accident given DUI') =", table.loc['DUI', 'Accident'] / table.loc['DUI', 'All'] )
```

Accident_Status	Accident	No accident	All
DUI_Status			
DUI	70	130	200
Non-DUI	30	770	800
All	100	900	1000

$$P(\text{DUI}) = 0.2$$

$$P(\text{'DUI and Accident'}) = 0.07$$

$$P(\text{'DUI and No accident'}) = 0.13$$

$$P(\text{'Non-DUI and Accident'}) = 0.03$$

$$P(\text{'DUI given Accident'}) = 0.7$$

$$P(\text{'Accident given DUI'}) = 0.35$$